

ArtConnect Pro

A Full-Stack Relational Database Application

Course	TI603I — Databases 2		
Year	2025–2026		
Team	Noah DELHI	Beya HENI	

Contents

Introduction	3
1 Functional Analysis	4
1.1 Application Overview	4
1.2 User Roles	4
1.3 Use Case Diagram	6
1.4 Class Diagram — Domain Model	7
2 Database Design	9
2.1 Entities and Attributes	9
2.2 Conceptual Data Model (CDM)	10
2.3 Logical Data Model (LDM)	11
2.4 Normalisation to 3NF	11
2.4.1 First Normal Form (1NF)	11
2.4.2 Second Normal Form (2NF)	11
2.4.3 Third Normal Form (3NF)	12
2.5 SQL Schema	12
3 Database Implementation and Advanced Features	14
3.1 Sample Data	14
3.2 Views	15
3.2.1 v_artist_portfolio — Query Simplification	15
3.2.2 v_workshop_availability — Hiding Complexity	15
3.2.3 v_member_activity — Security	16
3.2.4 Other Views	16
3.3 Indexes	16
3.4 Triggers	16
3.4.1 trg_check_workshop_capacity — Overbooking Prevention	17
3.4.2 trg_audit_artwork_status — Audit Trail	17
3.4.3 Other Triggers	18
3.5 Stored Procedures and Functions	18
3.5.1 fn_get_workshop_available_spots	18
3.5.2 fn_get_artist_average_rating	18
3.5.3 sp_create_workshop_with_artist	18
3.5.4 sp_register_member_to_workshops	19
3.5.5 sp_generate_artist_report	19

3.6	Transactions	19
3.6.1	Scenario 1 — Successful Multi-Operation Registration	19
3.6.2	Scenario 2 — ROLLBACK on Full Workshop	20
4	Application Architecture and JDBC Integration	21
4.1	3-Tier Layered Architecture	21
4.2	Dependency Injection — ServiceProvider	22
4.3	Database Connection — HikariCP	22
4.4	DAO Pattern — Example: JdbcArtistDao	22
4.5	UI Layer — Controllers	23
4.6	Application Screenshots	24
5	Tests and Validation	25
5.1	Artist CRUD Scenarios	25
5.2	Artwork CRUD Scenarios	25
5.3	Workshop and Trigger Scenarios	26
6	Challenges and Solutions	27
6.1	UTF-8 Encoding Corruption	27
6.2	Artist Discipline Filter Bug	27
6.3	Artworks Tab Stale Data After Cascade Delete	27
6.4	Hardcoded Foreign Key IDs in Data Script	28
6.5	LaTeX Blank Page Issue	28
	Conclusion	29
A	Project File Structure	30
B	SQL Execution Order	31

Introduction

ArtConnect Pro is a community platform designed to connect artists, showcase their works, and organise cultural events such as exhibitions and workshops. This project was developed as part of the *TI603I — Databases 2* course at EFREI Paris.

The initial codebase provided a Java application with in-memory data and no real persistence. Over five supervised steps, the team:

1. Analysed the application and defined its functional scope;
2. Designed a normalised relational data model (CDM, LDM, 3NF);
3. Implemented the MySQL database with advanced SQL objects;
4. Integrated the database into the Java application via JDBC;
5. Finalised, tested, and documented the complete system.

Technology Stack

Language	Java 23
UI Framework	JavaFX 23 + FXML
Database	MySQL 8 (utf8mb4)
Connectivity	JDBC 4 — HikariCP connection pool
Build Tool	Apache Maven
Architecture	3-tier layered (UI / Service / DAO)

1. Functional Analysis

1.1 Application Overview

The ArtConnect application is organised around **seven tabs**, each providing a distinct view of the platform's data.

Tab	Features
Discover	Featured exhibitions (cards), upcoming workshops. Landing page of the app.
Artists	List all artists · Search by name · Filter by discipline · CRUD (Add, Edit, Delete)
Artworks	List all artworks · Add artwork · Mark as Sold / For Sale / Exhibited · Delete
Galleries	List galleries with rating and contact info · Browse hosted exhibitions
Exhibitions	List exhibitions · Filter by gallery / date range · View curators and themes
Workshops	List workshops with level, instructor, availability · Book / Cancel
Community	List members · View bookings and reviews · Filter by membership type

1.2 User Roles

Two roles were identified, even though the current application has no authentication layer.

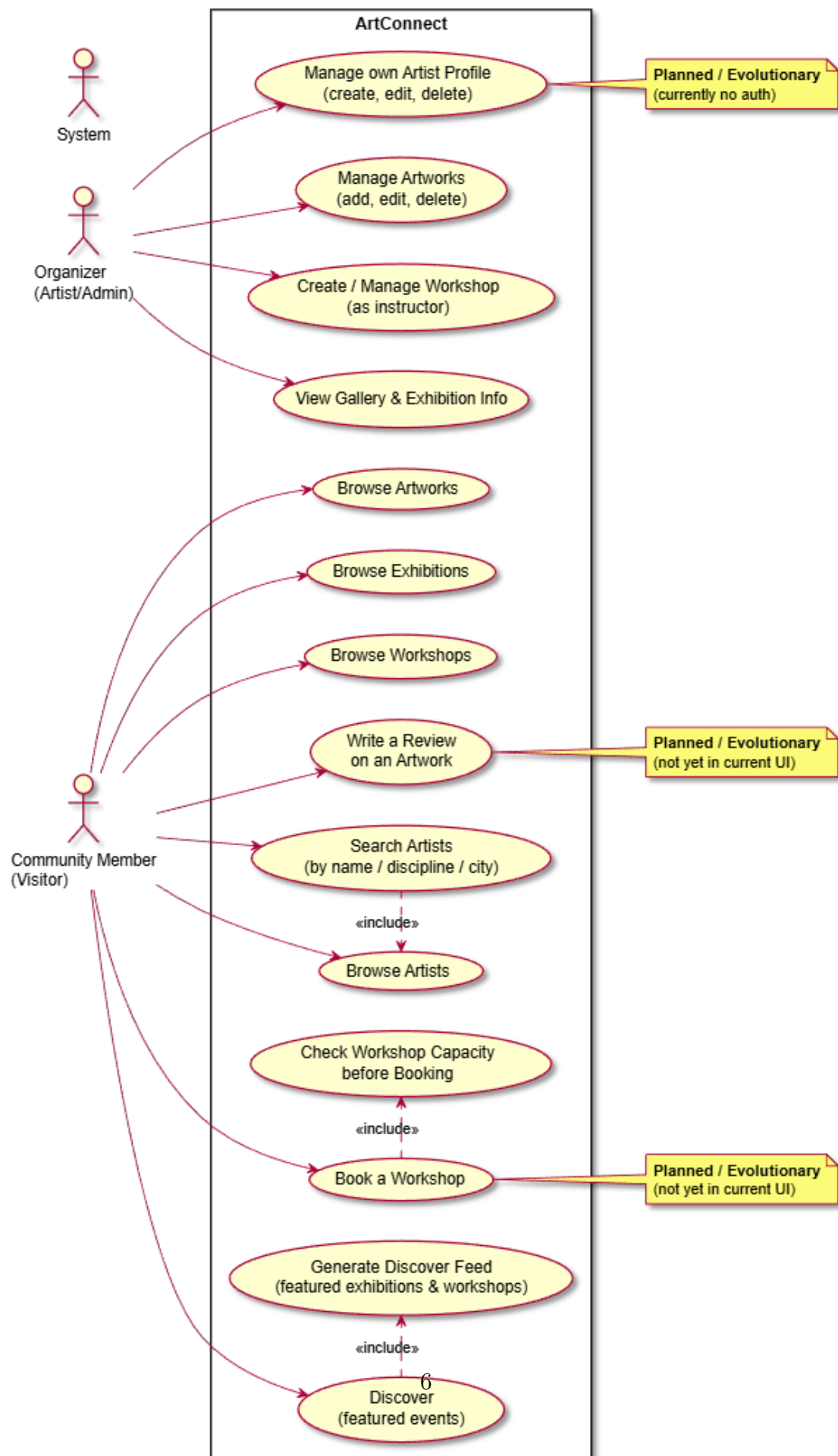
Role 1 — Organiser (Artist / Admin)

Creates and manages their artist profile, adds and modifies artworks, proposes workshops as instructor, and participates in gallery exhibitions.

Role 2 — Community Member (Visitor)

Browses artists, artworks, exhibitions, and workshops. Books workshop spots. Writes reviews (rating 1–5 + comment) on artworks. Has favourite disciplines.

1.3 Use Case Diagram



Legend:

- **Present in current UI:** Browse/search artists, artworks, exhibitions, galleries, workshops, community, discover feed, CRUD on artists and artworks, book/cancel workshops.
- **Planned (evolutionary):** User authentication, member-submitted reviews, full artist self-service profile.

1.4 Class Diagram — Domain Model

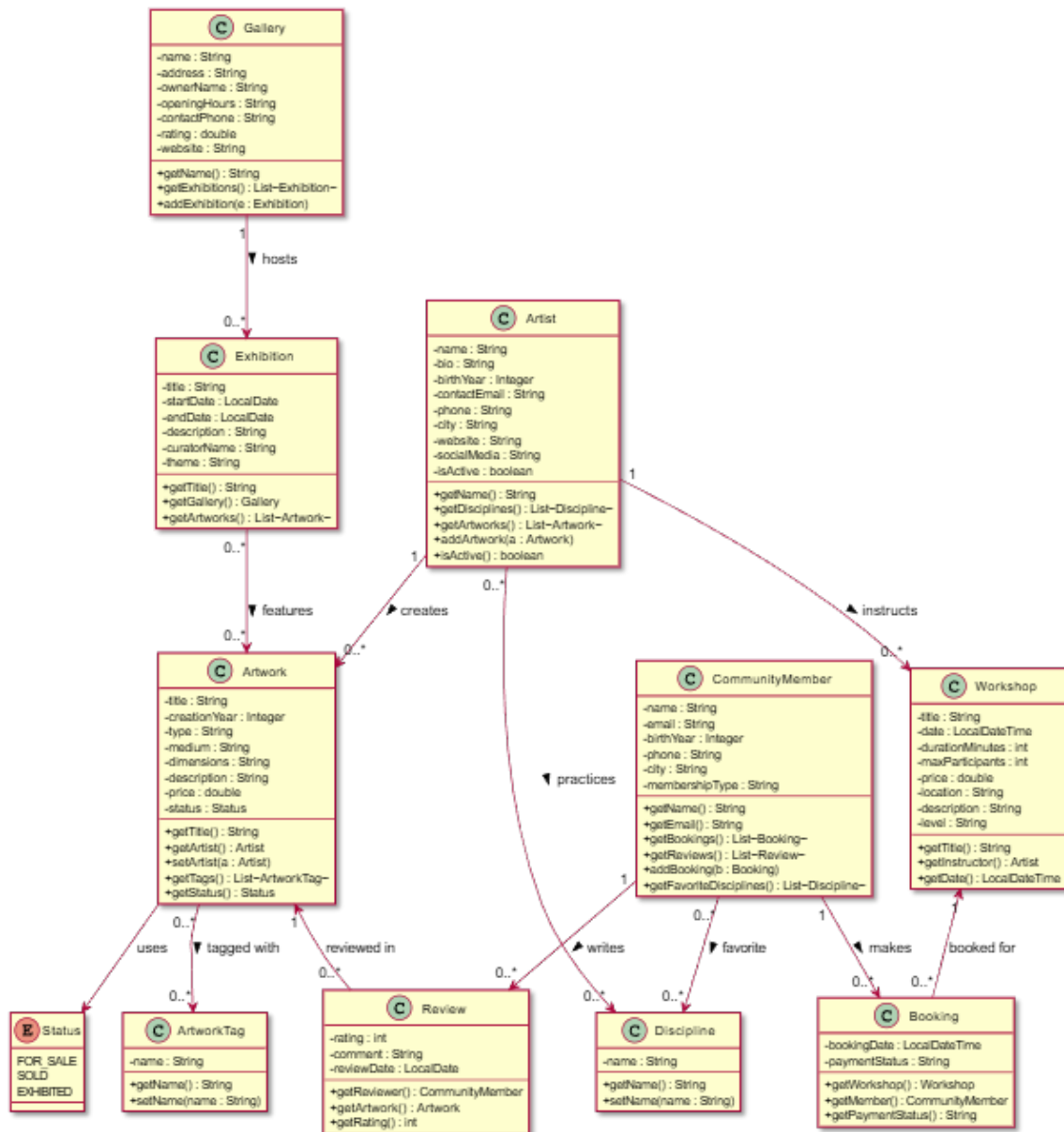


Figure 1.2: Domain model classes

The ten domain classes (Artist, Artwork, Gallery, Exhibition, Workshop, CommunityMember, Booking, Review, Discipline, ArtworkTag) directly map to the database tables described

in Chapter 2.

2. Database Design

2.1 Entities and Attributes

Entity	Main Attributes	Identifier
Artist	name, bio, birth_year, contact_email, phone, city, website, social_media, is_active	artist_id (PK)
Artwork	title, creation_year, type, medium, dimensions, description, price, status	artwork_id (PK)
Gallery	name, address, owner_name, opening_hours, contact_phone, rating, website	gallery_id (PK)
Exhibition	title, start_date, end_date, description, curator_name, theme	exhibition_id (PK)
Workshop	title, date, duration_minutes, max_participants, price, location, description, level	workshop_id (PK)
CommunityMember	name, email, birth_year, phone, city, membership_type	member_id (PK)
Booking	booking_date, payment_status	booking_id (PK)
Review	rating (1–5), comment, review_date	review_id (PK)
Discipline	name (UNIQUE)	discipline_id (PK)
ArtworkTag	name (UNIQUE)	tag_id (PK)

2.2 Conceptual Data Model (CDM)

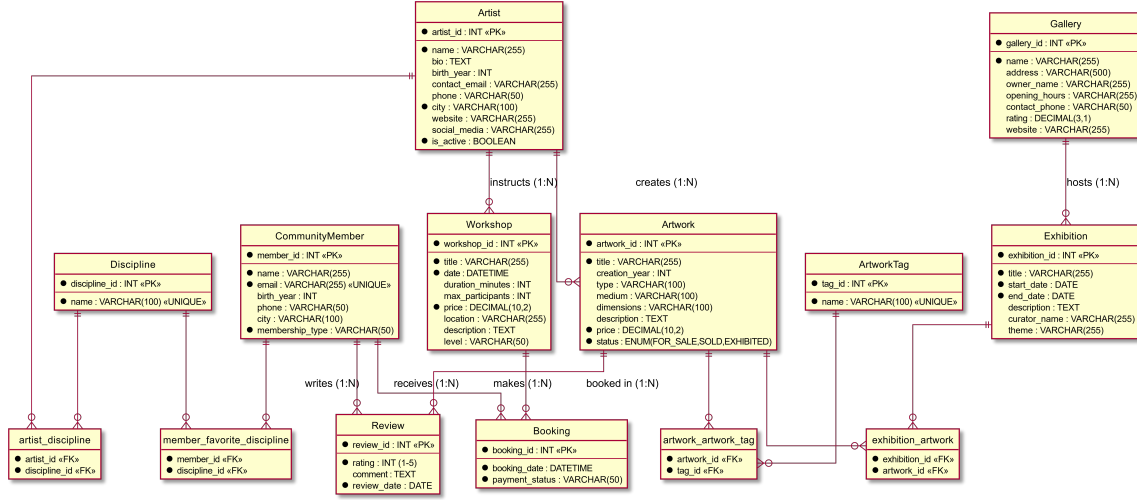


Figure 2.1: Entity-Relationship Diagram — ArtConnect

Key relationships:

- **Artist** → **Artwork** (1:N) — one artist creates many artworks; ON DELETE CASCADE.
- **Artist** **Discipline** (N:M) — via junction table `artist_discipline`.
- **Artist** → **Workshop** (1:N, ON DELETE RESTRICT) — an artist instructs workshops; cannot be deleted while they instruct one.
- **Gallery** → **Exhibition** (1:N) — a gallery hosts many exhibitions.
- **Exhibition** **Artwork** (N:M) — via `exhibition_artwork`; same artwork can appear in multiple exhibitions.
- **Workshop** → **Booking** (1:N) — one workshop has many bookings.
- **CommunityMember** → **Booking** / **Review** (1:N).
- **CommunityMember** **Discipline** (N:M) — favourite disciplines.

2.3 Logical Data Model (LDM)

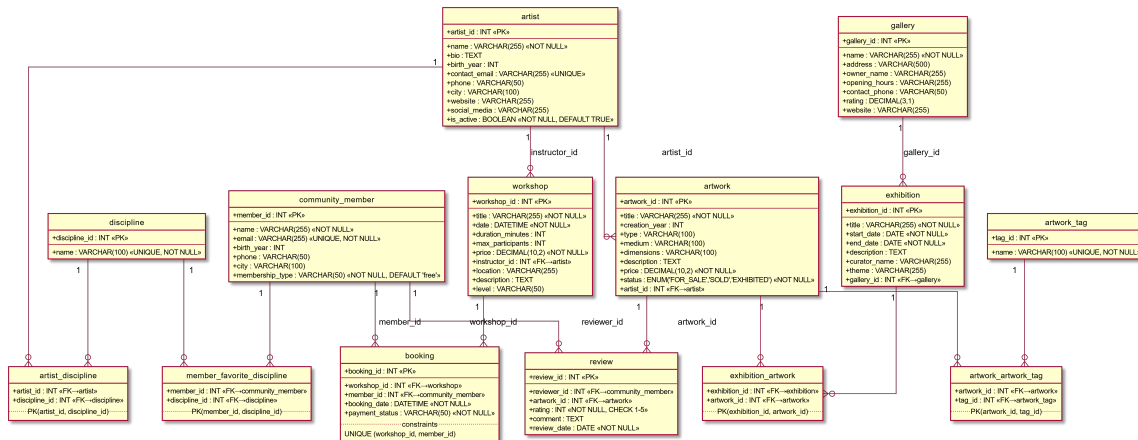


Figure 2.2: Logical Data Model

The LDM adds primary keys (all surrogate INT AUTO_INCREMENT), foreign keys, data types, and constraints. Four junction tables resolve the N:M relationships:

Junction Table	Resolves
artist_discipline	Artist Discipline
member_favorite_discipline	CommunityMember Discipline
exhibition_artwork	Exhibition Artwork
artwork_artwork_tag	Artwork ArtworkTag

2.4 Normalisation to 3NF

2.4.1 First Normal Form (1NF)

Rule: Each column must contain atomic values; no repeating groups.

Applied: The Java model classes used `List<Discipline>`, `List<ArtworkTag>`, `List<Artwork>` (on Exhibition), etc. All multi-valued attributes were extracted into separate junction tables with one value per row.

Result: All tables contain only atomic values.

2.4.2 Second Normal Form (2NF)

Rule: Every non-key attribute must depend on the *whole* primary key (applies to composite PKs).

Applied: Junction tables (`artist_discipline`, etc.) have composite PKs but *no non-key attributes*, so 2NF is trivially satisfied. `booking` was given a surrogate `booking_id` so that `booking_date` and `payment_status` depend on a single key; the pair (`workshop_id`, `member_id`) is still enforced by a UNIQUE constraint.

Result: 2NF satisfied for all tables.

2.4.3 Third Normal Form (3NF)

Rule: No non-key attribute may transitively depend on the primary key.

Table	Analysis	Status
artist	All attributes depend directly on artist_id.	
artwork	artist_id is a FK, not derived. status depends on artwork_id only.	
gallery	rating, address depend on gallery_id.	
exhibition	curator_name depends on exhibition_id, not on gallery_id.	
workshop	level and location depend on workshop_id. instructor_id is a FK.	
community_member	membership_type depends on member_id, not transitively on city.	
booking	payment_status depends on booking_id directly.	
review	rating and comment depend on review_id, not on artwork_id.	
Junction tables	No non-key attributes at all.	

Result: No transitive dependencies exist. All tables are in 3NF.

2.5 SQL Schema

The complete schema is contained in `sql/artconnect_schema.sql`. Below is an extract illustrating the key design decisions.

```

1 DROP DATABASE IF EXISTS artconnect;
2 CREATE DATABASE artconnect
3     CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
4 USE artconnect;
5
6 CREATE TABLE artist (
7     artist_id    INT NOT NULL AUTO_INCREMENT,
8     name         VARCHAR(255) NOT NULL,
9     bio          TEXT,
10    birth_year   INT,
11    contact_email VARCHAR(255),
12    city          VARCHAR(100),
13    is_active     BOOLEAN NOT NULL DEFAULT TRUE,
14    CONSTRAINT pk_artist PRIMARY KEY (artist_id),
15    CONSTRAINT uq_artist_email UNIQUE (contact_email)
16 );
17
18 CREATE TABLE artwork (

```

```
19 artwork_id    INT NOT NULL AUTO_INCREMENT ,
20 title         VARCHAR(255) NOT NULL ,
21 price         DECIMAL(10,2) NOT NULL DEFAULT 0.00 ,
22 status        ENUM('FOR_SALE','SOLD','EXHIBITED') NOT NULL
                DEFAULT 'FOR_SALE' ,
23 artist_id     INT NOT NULL ,
24 CONSTRAINT pk_artwork      PRIMARY KEY (artwork_id),
25 CONSTRAINT fk_artwork_artist
26         FOREIGN KEY (artist_id) REFERENCES artist(artist_id)
27         ON DELETE CASCADE ON UPDATE CASCADE
28 );
```

Listing 2.1: Schema extract — artist and artwork tables

Key Design Decisions

- **utf8mb4** character set ensures correct storage of all Unicode characters (accents, special characters).
- **ON DELETE CASCADE** on `artwork.artist_id`: deleting an artist automatically removes all their artworks.
- **ON DELETE RESTRICT** on `workshop.instructor_id`: an artist who instructs a workshop cannot be deleted until the workshop is removed first.
- **ENUM** type for `artwork.status` enforces valid values at the database level.
- **CHECK constraints** on `exhibition` (`end_date ≥ start_date`), `review` (rating 1–5), and `booking` (`payment_status IN ...`).
- **DROP DATABASE IF EXISTS** at the top of the schema ensures clean re-execution on any machine.

3. Database Implementation and Advanced Features

3.1 Sample Data

The database is populated with realistic data generated with the help of an AI assistant. The following prompt was used:

Prompt used for data generation

“Generate realistic sample data for the ArtConnect database covering all 10 entities and 4 junction tables. Include 5 disciplines, 8 artwork tags, 6 artists from different cities (one inactive), 15 artworks with all 3 status values, 4 galleries, 5 exhibitions, 6 workshops at different levels, 8 community members (mix of free/premium), 12 bookings with varied payment statuses, and 10 reviews. Include cross-participations (artworks appearing in multiple exhibitions). The data should tell a coherent story and demonstrate interesting query results.”

Entity	Count	Notes
discipline	5	Painting, Sculpture, Photography, Digital Art, Mixed Media
artwork_tag	8	Abstract, Contemporary, Portrait, Landscape, Minimalist, Surrealist, Pop Art, Expressionist
artist	11	From Paris, Rome, Tokyo, Dakar, Stockholm, São Paulo, Accra, Gothenburg, Barcelona, Shanghai (1 inactive)
artwork	20	Original 15 + 5 for new artists; status: FOR_SALE, SOLD, EXHIBITED
gallery	4	Paris, Rome, Stockholm, London
exhibition	5	Dates spanning 2024-09 to 2025-09
workshop	6	2 Beginner, 2 Intermediate, 2 Advanced
community_member	8	4 free, 4 premium
booking	12	8 PAID, 2 PENDING, 2 CANCELLED
review	10	Ratings: 3×5★, 5×4★, 1×3★

Cross-participations: Artworks 3 (*Urban Fragments #7*) and 7 (*Neon Dreams*:

Shibuya) each appear in two different exhibitions, demonstrating the N:M `exhibition_artwork` relationship.

3.2 Views

Five views were created covering query simplification and security objectives.

3.2.1 `v_artist_portfolio` — Query Simplification

Avoids a complex multi-join query every time the Artists tab loads. Aggregates discipline names, artwork count, and average price per artist.

```

1 CREATE OR REPLACE VIEW v_artist_portfolio AS
2 SELECT
3     a.artist_id,
4     a.name AS artist_name,
5     a.city,
6     a.is_active,
7     GROUP_CONCAT(DISTINCT d.name ORDER BY d.name SEPARATOR ', '
8                     )
9                     AS disciplines,
10    COUNT(DISTINCT aw.artwork_id) AS
11    artwork_count,
12    COALESCE(ROUND(AVG(aw.price), 2), 0) AS avg_price
13 FROM artist a
14 LEFT JOIN artist_discipline ad ON a.artist_id = ad.artist_id
15 LEFT JOIN discipline d ON ad.discipline_id = d.
16 discipline_id
17 LEFT JOIN artwork aw ON a.artist_id = aw.artist_id
18 GROUP BY a.artist_id, a.name, a.city, a.is_active;

```

Listing 3.1: `v_artist_portfolio`

3.2.2 `v_workshop_availability` — Hiding Complexity

Encapsulates the available-spots calculation so the UI queries this view directly.

```

1 CREATE OR REPLACE VIEW v_workshop_availability AS
2 SELECT
3     w.workshop_id,
4     w.title AS workshop_title,
5     w.max_participants,
6     COUNT(CASE WHEN b.payment_status IN ('PAID', 'PENDING') THEN
7             1 END)
8             AS confirmed_bookings,
9     w.max_participants
10    - COUNT(CASE WHEN b.payment_status IN ('PAID', 'PENDING')
11            THEN 1 END)
12            AS available_spots
13 FROM workshop w
14 JOIN artist a ON w.instructor_id = a.artist_id

```



```

13 LEFT JOIN booking b ON w.workshop_id = b.workshop_id
14 GROUP BY w.workshop_id, ...;

```

Listing 3.2: v_workshop_availability (extract)

3.2.3 v_member_activity — Security

Provides member activity summary **without exposing sensitive columns** (email, phone).

3.2.4 Other Views

View	Objective	Content
v_exhibition_details	Simplification	Exhibition + gallery + duration + artwork count
v_artwork_catalog	Simplification	Full catalog with artist name, tags, avg rating

3.3 Indexes

Six indexes were created targeting the most frequent access patterns.

Index	Column(s)	Justification
idx_artwork_artist	artwork(artist_id)	Used in virtually every artwork–artist JOIN
idx_artwork_status	artwork(status)	Frequent WHERE status = 'FOR_SALE' filters
idx_exhibition_dates	exhibition(start_date, end_date)	Composite for date-range queries
idx_review_artwork	review(artwork_id)	Accelerates AVG(rating) in v_artwork_catalog
idx_booking_workshop	booking(workshop_id)	Used by the capacity trigger
idx_workshop_date	workshop(date)	ORDER BY and range scans on upcoming workshops

Note on Implicit Indexes

Primary key columns and UNIQUE constraints already create implicit B-tree indexes in MySQL InnoDB. The six indexes above only target **non-key, non-unique** columns that are frequently used in WHERE, JOIN, or ORDER BY clauses.

3.4 Triggers

Four triggers implement automated database-level enforcement.

3.4.1 trg_check_workshop_capacity — Overbooking Prevention

```
1 CREATE TRIGGER trg_check_workshop_capacity
2 BEFORE INSERT ON booking
3 FOR EACH ROW
4 BEGIN
5     DECLARE current_count INT;
6     DECLARE max_spots     INT;
7
8     SELECT COUNT(*) INTO current_count
9     FROM booking
10    WHERE workshop_id = NEW.workshop_id
11          AND payment_status IN ('PAID', 'PENDING');
12
13     SELECT max_participants INTO max_spots
14     FROM workshop WHERE workshop_id = NEW.workshop_id;
15
16     IF max_spots IS NOT NULL AND current_count >= max_spots
17     THEN
18         SIGNAL SQLSTATE '45000'
19         SET MESSAGE_TEXT = 'Workshop is full: no more spots
20                             available.';
21     END IF;
22 END //
```

Listing 3.3: Capacity check trigger

3.4.2 trg_audit_artwork_status — Audit Trail

Logs every artwork status change (e.g. FOR_SALE → SOLD) to the audit_log table for full traceability.

```
1 CREATE TRIGGER trg_audit_artwork_status
2 AFTER UPDATE ON artwork
3 FOR EACH ROW
4 BEGIN
5     IF OLD.status <> NEW.status THEN
6         INSERT INTO audit_log
7             (table_name, record_id, action, old_value,
8              new_value)
9         VALUES ('artwork', NEW.artwork_id, 'STATUS_CHANGE',
10                OLD.status, NEW.status);
11     END IF;
12 END //
```

Listing 3.4: Audit trigger

3.4.3 Other Triggers

Trigger	Event	Purpose
trg_check_exhibition_date_consistency	BEFORE UPDATE ON exhibition	Ensures end_date ≥ start_date
trg_set_artwork_exhibited	AFTER INSERT ON exhibition_artwork	Auto-sets artwork to EXHIBITED when linked to an active exhibition

3.5 Stored Procedures and Functions

3.5.1 fn_get_workshop_available_spots

```

1 CREATE FUNCTION fn_get_workshop_available_spots(p_workshop_id
  INT)
2 RETURNS INT
3 DETERMINISTIC READS SQL DATA
4 BEGIN
5     DECLARE v_max      INT;
6     DECLARE v_booked INT;
7
8     SELECT max_participants INTO v_max
9     FROM workshop WHERE workshop_id = p_workshop_id;
10
11     IF v_max IS NULL THEN RETURN NULL; END IF;
12
13     SELECT COUNT(*) INTO v_booked
14     FROM booking
15     WHERE workshop_id = p_workshop_id
16         AND payment_status IN ('PAID', 'PENDING');
17
18     RETURN v_max - v_booked;
19 END //
```

Listing 3.5: Available spots function

Usage: `SELECT fn_get_workshop_available_spots(1);`

3.5.2 fn_get_artist_average_rating

Returns the average review rating across all artworks of a given artist. Returns NULL if no reviews exist.

Usage: `SELECT fn_get_artist_average_rating(1);`

3.5.3 sp_create_workshop_with_artist

Creates a new workshop. If the specified instructor (identified by email) does not already exist, creates the artist record first and uses the new `artist_id` as `instructor_id`.

```
1 CREATE PROCEDURE sp_create_workshop_with_artist(  
2     IN p_title VARCHAR(255), IN p_date DATETIME, ...,  
3     IN p_instructor_email VARCHAR(255),  
4     IN p_instructor_name  VARCHAR(255)  
5 )  
6 BEGIN  
7     DECLARE v_artist_id INT;  
8     SELECT artist_id INTO v_artist_id  
9     FROM artist WHERE contact_email = p_instructor_email LIMIT  
10        1;  
11  
12     IF v_artist_id IS NULL THEN  
13         INSERT INTO artist (name, contact_email, is_active)  
14         VALUES (p_instructor_name, p_instructor_email, TRUE);  
15         SET v_artist_id = LAST_INSERT_ID();  
16     END IF;  
17  
18     INSERT INTO workshop (title, date, ..., instructor_id)  
19     VALUES (p_title, p_date, ..., v_artist_id);  
20 END //
```

Listing 3.6: sp_create_workshop_with_artist (extract)

3.5.4 sp_register_member_to_workshops

Registers a member to one or more workshops at once, accepting a comma-separated list of workshop IDs. Uses a loop to insert bookings one by one, skipping workshops that are already booked or full.

Usage: CALL sp_register_member_to_workshops(1, '1,3,5');

3.5.5 sp_generate_artist_report

Displays a comprehensive dossier for a given artist: basic info with disciplines, all artworks with status and ratings, exhibitions featuring their works, and workshops they instruct.

Usage: CALL sp_generate_artist_report(1);

3.6 Transactions

Two transactional scenarios are provided in sql/artconnect_transactions.sql.

3.6.1 Scenario 1 — Successful Multi-Operation Registration

A new premium member joins ArtConnect, selects two favourite disciplines, books three workshops, and writes two reviews — all atomically.

```
1 START TRANSACTION;  
2  
3 INSERT INTO community_member (name, email, birth_year, city,  
    membership_type)
```

```
4 VALUES ('Lea Fontaine', 'lea.fontaine@email.com', 1994, '
   Marseille', 'premium');
5
6 SET @new_member_id = LAST_INSERT_ID();
7
8 INSERT INTO member_favorite_discipline (member_id,
   discipline_id)
9 VALUES (@new_member_id, 1), (@new_member_id, 3);
10
11 INSERT INTO booking (workshop_id, member_id, booking_date,
   payment_status)
12 VALUES (1, @new_member_id, NOW(), 'PAID');
13 -- ... two more bookings ...
14 -- ... two reviews ...
15
16 COMMIT;
```

Listing 3.7: Transaction Scenario 1 (extract)

3.6.2 Scenario 2 — ROLLBACK on Full Workshop

A stored procedure `sp_demo_atomic_registration` attempts to book a member into two workshops, one of which is artificially filled. The capacity trigger fires a `SIGNAL`, the `EXIT HANDLER FOR SQLEXCEPTION` catches it, and the entire transaction is rolled back — demonstrating atomicity.

```
1 CREATE PROCEDURE sp_demo_atomic_registration()
2 BEGIN
3     DECLARE EXIT HANDLER FOR SQLEXCEPTION
4     BEGIN
5         ROLLBACK;
6         SELECT 'Transaction ROLLED BACK.' AS result;
7     END;
8
9     START TRANSACTION;
10    INSERT INTO community_member (...) VALUES ('Marc Lefevre',
   ...);
11    SET v_member_id = LAST_INSERT_ID();
12    INSERT INTO booking (...) VALUES (5, v_member_id, NOW(), '
   PENDING'); -- OK
13    INSERT INTO booking (...) VALUES (2, v_member_id, NOW(), '
   PENDING'); -- FAILS
14    COMMIT; -- never reached
15 END //
```

Listing 3.8: ROLLBACK scenario with handler (extract)

4. Application Architecture and JDBC Integration

4.1 3-Tier Layered Architecture

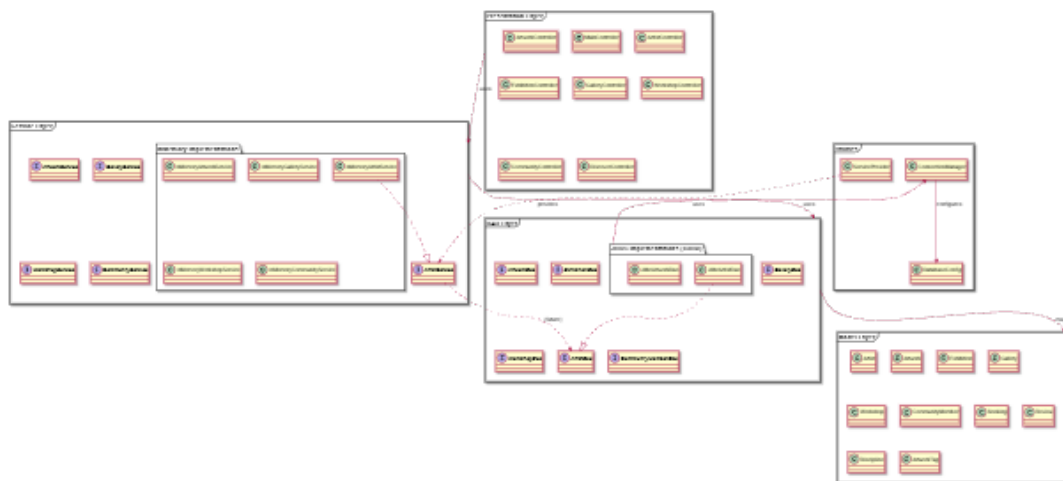


Figure 4.1: 3-tier layered architecture

The application strictly separates concerns into three layers:

Layer Summary

Layer	Technology	Responsibility
UI Layer	JavaFX 23 + FXML	Tab controllers, dialogs, status labels
Service Layer	Java interfaces + JDBC impls	Business logic, validation, transactions
DAO Layer	JDBC PreparedStatement	SQL execution, result mapping, connection management

4.2 Dependency Injection — ServiceProvider

A `ServiceProvider` singleton acts as a manual dependency injection container, providing instances of each service to the controllers.

```
1 public class ServiceProvider {
2     private static ArtistService artistService;
3     private static ArtworkService artworkService;
4
5     public static ArtistService getArtistService() {
6         if (artistService == null)
7             artistService = new JdbcArtistService();
8         return artistService;
9     }
10    // same pattern for all services
11 }
```

Listing 4.1: ServiceProvider pattern

This design allows swapping `JdbcArtistService` for `InMemoryArtistService` without touching the UI layer — useful for offline testing.

4.3 Database Connection — HikariCP

Connection pooling is managed by HikariCP, configured via `database.properties`.

```
1 HikariConfig config = new HikariConfig();
2 config.setJdbcUrl(props.getProperty("db.url")); // includes
3     charset params
4 config.setUsername(props.getProperty("db.user"));
5 config.setPassword(props.getProperty("db.password"));
6 config.setMinimumIdle(2);
7 config.setMaximumPoolSize(10);
8 dataSource = new HikariDataSource(config);
```

Listing 4.2: ConnectionManager — HikariCP setup

The JDBC URL includes `useUnicode=true&characterEncoding=UTF-8` to ensure correct encoding between the Java layer and MySQL.

4.4 DAO Pattern — Example: JdbcArtistDao

Each DAO implements an interface and uses `PreparedStatement` with `try-with-resources` for safe resource management.

```
1 @Override
2 public void delete(String artistName) {
3     String sql = "DELETE FROM artist WHERE name = ?";
4     try (Connection conn = ConnectionManager.getConnection();
5         PreparedStatement ps = conn.prepareStatement(sql)) {
6         ps.setString(1, artistName);
7         ps.executeUpdate();
8     }
```

```

8      // ON DELETE CASCADE removes all artworks,
      //      artist_discipline rows
9    } catch (SQLException e) {
10      throw new RuntimeException("Error deleting artist", e);
11    }
12 }

```

Listing 4.3: JdbcArtistDao.delete (CASCADE demonstration)

Artist creation uses a manual transaction to atomically insert the artist and their discipline associations:

```

1  conn.setAutoCommit(false);
2  try {
3      // 1. Insert artist row
4      // 2. Retrieve generated artist_id
5      // 3. Insert rows in artist_discipline
6      conn.commit();
7  } catch (SQLException e) {
8      conn.rollback();
9      throw e;
10 } finally {
11     conn.setAutoCommit(true);
12 }

```

Listing 4.4: Transactional artist creation

4.5 UI Layer — Controllers

Each tab has a dedicated JavaFX controller. Controllers are thin: they collect user input, call the service, and update the UI.

Controller	Service Used	Actions
ArtistController	ArtistService	Search, Add (with discipline), Edit, I
ArtworkController	ArtworkService + ArtistService	Add, Delete, Mark Sold, Mark For S
WorkshopController	WorkshopService + CommunityService	Book, Cancel
ExhibitionController	GalleryService	Browse, filter by gallery
GalleryController	GalleryService	Browse with exhibition list
CommunityController	CommunityService	Browse, view bookings and reviews
DiscoverController	ArtworkService + WorkshopService	Featured cards

The `MainController` listens for tab-selection changes and triggers a data refresh when the Artworks tab is activated — ensuring that artworks deleted via CASCADE (after an artist deletion) are immediately removed from the view.

4.6 Application Screenshots

Name	City	Email	Birth Year
Elena Monseau	Paris	elena.monseau@gmail.com	1955
Marcio Beltrini	Rome	marcio.beltrini@gmail.com	1978
Yuki Tanaka	Tokyo	yuki.tanaka@gmail.com	1992
Amara Diello	Dakar	amara.diello@gmail.com	1990
Lucas Engdren	Stockholm	lucas.engdren@gmail.com	1982
Sophie Dubois	Brussels	sophie.dubois@gmail.com	1975
Isabela Carvalho	São Paulo	isabela.carvalho@gmail.com	1994
Kwame Asante	Accra	kwame.asante@gmail.com	1992
Nora Lindgren	Göteborg	nora.lindgren@gmail.com	1998
Rafael Torres	Barcelona	rafael.torres@gmail.com	1973
Mei Lin	Shanghai	mei.lin@gmail.com	1988

(a) Artists tab — search and CRUD

Title	Artist	Type	Price	Status
Survivor over Montmartre	Elena Monseau	Painting	4200.0	FOR SALE
Lavender Fields Forever	Elena Monseau	Painting	3200.0	SOLD
Urban Fragments #7	Elena Monseau	Mixed Media	2800.0	EXHIBITED
The Thinker Reimagined	Marcio Beltrini	Sculpture	15000.0	EXHIBITED
Hanbury in Rome	Marcio Beltrini	Sculpture	8500.0	FOR SALE
Iron Winds	Marcio Beltrini	Sculpture	22000.0	SOLD
Neon Dreams: Shinjuku	Yuki Tanaka	Photography	1800.0	FOR SALE
Digital Garden #3	Yuki Tanaka	Digital Art	2300.0	SOLD
Tokyo Solitude	Yuki Tanaka	Photography	1200.0	FOR SALE
Roots and Routes	Amara Diello	Mixed Media	5500.0	SOLD
Market Day in Dakar	Amara Diello	Painting	3800.0	FOR SALE
Women Memories	Amara Diello	Mixed Media	2400.0	SOLD
Composition in Blue #12	Lucas Engdren	Painting	1900.0	FOR SALE
Silent Grid	Lucas Engdren	Painting	2100.0	FOR SALE
Cherry in the Mist	Sophie Dubois	Photography	950.0	SOLD
Corridor of Culture	Isabela Carvalho	Painting	5200.0	EXHIBITED
Avantiers Speak	Kwame Asante	Sculpture	9800.0	SOLD
The Dreaming Forest	Nora Lindgren	Digital Art	1400.0	FOR SALE
The Last Shepherd	Rafael Torres	Photography	2300.0	EXHIBITED
Mountain Mist at Dawn	Mei Lin	Painting	4100.0	SOLD

(b) Artworks tab — status management

Title	Instructor	Date	Price	Level	Available Spots
Introduction to Oil Painting	Elena Monseau	2025-03-15T11:00	75.0	Beginner	12
Abstract Sculpture Workshop	Marcio Beltrini	2025-04-10T16:00	120.0	Intermediate	8
Night Photography Masterclass	Yuki Tanaka	2025-05-20T21:00	60.0	Intermediate	15
Mixed Media Collage Intensive	Amara Diello	2025-06-03T11:00	95.0	Advanced	10
Geometric Abstraction Theory & P.	Lucas Engdren	2025-07-12T10:00	45.0	Beginner	20
Advanced Color Theory for Artists	Elena Monseau	2025-08-22T15:00	85.0	Advanced	10

(a) Workshops tab — booking

Featured Exhibitions & Content

- FEATURED EXHIBITION**
Visions of Tomorrow
Theme: Futurism & Innovation
Gallery: Galerie Lumière
- FEATURED EXHIBITION**
Urban Pulse
Theme: Urban Life
Gallery: Galerie Lumière
- FEATURED EXHIBITION**
Sculpting the Modern World
Theme: Modern Sculpture
Gallery: Spazio Arte Moderna
- UPCOMING WORKSHOP**
Abstract Sculpture Workshop
Instructor: Marcio Beltrini
Price: \$120.0
- UPCOMING WORKSHOP**
Night Photography Masterclass
Instructor: Yuki Tanaka
Price: \$60.0
- UPCOMING WORKSHOP**
Introduction to Oil Painting
Instructor: Elena Monseau
Price: \$75.0

(b) Discover tab — featured content

Title	Gallery	Start Date	Theme
Visions of Tomorrow	Galerie Lumière	2024-09-01	Futurism & Innovation
Urban Pulse	Galerie Lumière	2025-06-01	Urban Life
Sculpting the Modern World	Spazio Arte Moderna	2024-10-15	Modern Sculpture
Light & Shadow: Nordic Perspectives	Nordic Art House	2025-01-10	Nordic Minimalism
Global Threads	Galerie Horizonte	2025-02-01	Cultural Identity

(a) Exhibitions tab

Title	Address	Location
Galerie Lumière - 15 Rue de Flandre	75001 Paris, France	(47.5, 2)
Spazio Arte Moderna - Via del Corso 18	00186 Rome, Italy	(43.5, 12)
Nordic Art House - Drottninggatan 43	111 21 Stockholm, Sweden	(59.3, 18)
Galerie Horizonte - 123 King Street, Covent Garden	London WC2E 8JQ, UK	(51.5, 1)

(b) Galleries tab

5. Tests and Validation

5.1 Artist CRUD Scenarios

Scenario	Expected Result	Status
Load all artists	List populated from MySQL via <code>getAllArtists()</code>	
Search artist by name	Filters using LIKE query on <code>artist.name</code>	
Filter by discipline	SQL WHERE on joined <code>discipline</code> table	
Add artist with discipline	INSERT into <code>artist</code> + row in <code>artist_discipline</code> (transactional)	
Edit artist (city change)	UPDATE <code>artist</code> row; disciplines refreshed (DELETE + re-INSERT)	
Delete artist	CASCADE removes all <code>artwork</code> , <code>artist_discipline</code> rows	

5.2 Artwork CRUD Scenarios

Scenario	Expected Result	Status
Load all artworks	JOIN <code>artwork</code> with <code>artist</code> to re-construct artist object	
Add artwork linked to artist	INSERT with subquery resolving <code>artist_id</code> by name	
New artwork status	Automatically set to EXHIBITED on creation	
Mark artwork as SOLD	<code>trg_audit_artwork_status</code> fires; row inserted in <code>audit_log</code>	
Delete artwork	Row removed; Artworks tab auto-refreshes via tab-selection listener	

Scenario	Expected Result	Status
Delete artist (cascade)	All artworks removed from DB; Artworks tab updates on next focus	

5.3 Workshop and Trigger Scenarios

Scenario	Expected Result	Status
Book a workshop (normal)	INSERT into booking; available_spots decrements in view	
Book a full workshop	trg_check_workshop_capacity fires SIGNAL; error shown in UI	
Cancel booking	DELETE from booking; available_spots increments	
Add artwork to active exhibition	trg_set_artwork_exhibited fires; status auto-set to EXHIBITED	
Update exhibition dates (end < start)	trg_check_exhibition_date_consistency blocks the update	

6. Challenges and Solutions

6.1 UTF-8 Encoding Corruption

Problem: Artist names containing special characters (e.g. “Lucas Engström”, “Léa Fontaine”) appeared corrupted in the application (?? or box-drawing characters).

Root cause: The JDBC connection string did not specify a character encoding. MySQL stored the bytes as received (Latin-1 or raw UTF-8 bytes), producing either 3F3F (literal ?) for lost data, or garbled multi-byte sequences for double-encoded data. This was diagnosed with:

```
1 SELECT name, HEX(name) FROM artist WHERE name LIKE 'Lucas%';
2 -- Result: 4C7563617320456E677374723F3F6D (3F = '?')
```

Solution: Added `useUnicode=true&characterEncoding=UTF-8` to the JDBC URL in `database.properties`. Corrupted rows were corrected with targeted UPDATE statements using `SET SQL_SAFE_UPDATES = 0`.

6.2 Artist Discipline Filter Bug

Problem: A newly added artist did not appear when filtering by discipline in the Artists tab.

Root cause: The Add Artist dialog had no discipline field. Artists were created with an empty `disciplines` list. The `searchArtists()` method performs a JOIN on `artist_discipline`, which excludes artists with no discipline entry.

Solution: Added a `ComboBox<Discipline>` to the Add Artist dialog. The selected discipline is passed to the artist object before the `createArtist()` call, which persists it in a transaction.

6.3 Artworks Tab Stale Data After Cascade Delete

Problem: Deleting an artist removed their artworks from the database (ON DELETE CASCADE) but the Artworks tab continued to show the deleted artworks.

Root cause: `ArtworkController.initialize()` is called only once when the FXML loads. The table data is never refreshed automatically.

Solution: Added a tab-selection listener in `MainController` that calls `artworksTabController.ref` whenever the Artworks tab is selected. The `artworksTabController` reference is injected via the `fx:id` mechanism of `fx:include`.

6.4 Hardcoded Foreign Key IDs in Data Script

Problem: The additional data INSERT statements used hardcoded `artist_id` values (7–11). Running the script on a machine with a different initial state caused foreign key constraint violations.

Solution: Replaced all hardcoded ID references with `SELECT`-based subqueries resolving entities by name:

```
1 INSERT INTO artist_discipline (artist_id, discipline_id)
2 SELECT a.artist_id, d.discipline_id
3 FROM artist a, discipline d
4 WHERE a.name = 'Isabela Carvalho' AND d.name = 'Painting';
```

6.5 LaTeX Blank Page Issue

Problem: The PDF report had an unwanted blank page caused by the `report` document class inserting a `\cleardoublepage` between chapters.

Solution: Added `\let\cleardoublepage\clearpage` in the preamble and defined a plain page style with no header rule.

Conclusion

The ArtConnect Pro project successfully delivers a complete database-backed application meeting all five project steps.

Database-level achievements:

- A fully normalised (3NF) MySQL schema with 11 tables, appropriate constraints, and utf8mb4 encoding.
- 4 triggers for automation and data integrity.
- 5 views for query simplification and security.
- 6 indexes for query optimisation.
- 3 stored procedures and 2 functions.
- 2 transactional scenarios demonstrating COMMIT and ROLLBACK.

Application-level achievements:

- Full replacement of in-memory services with JDBC-backed implementations.
- CRUD operations on artists and artworks (Add, Edit, Delete, status changes).
- Workshop booking and cancellation with capacity enforcement via trigger.
- Automatic tab refresh ensuring UI consistency after cascade deletions.

Key lesson: Placing business rules (capacity checks, audit logging, date validation) at the database level rather than in application code makes them enforcement-proof — they cannot be bypassed regardless of the client, whether it is the Java application, MySQL Workbench, or a direct SQL connection.

A. Project File Structure

```
PJ-DB-I1/
sql/
    artconnect_schema.sql      -- DROP + CREATE all tables
    artconnect_data.sql       -- 11 artists, 20 artworks, ...
    artconnect_triggers.sql    -- 4 triggers + audit_log table
    artconnect_views_indexes.sql -- 5 views + 6 indexes
    artconnect_procedures.sql  -- 2 functions + 3 procedures
    artconnect_transactions.sql -- 2 transactional scenarios
docs/
    FinalReport.tex           -- This document
diagrams/
    Conceptual Data Model- ERD.png
    LDM Diagram.png
    Architecture - Layer Diagram.png
    Model Layer - Domain Classes.png
    Use Case Diagram.png
screenshots/
    screenshot_*.png
ArtConnectPro-App/
    src/main/java/com/project/artconnect/
        model/                -- Domain classes
        persistence/          -- JdbcXxxDao implementations
        service/              -- Service interfaces
        service/impl/         -- JdbcXxxService + InMemoryXxxService
        ui/                   -- JavaFX controllers
        util/                 -- ServiceProvider, ConnectionManager
```

B. SQL Execution Order

To deploy ArtConnect Pro on a new machine, execute the scripts in the following order:

1. `artconnect_schema.sql` — drops and recreates the database and all tables.
2. `artconnect_data.sql` — inserts all sample data.
3. `artconnect_triggers.sql` — creates the `audit_log` table and all triggers.
4. `artconnect_views_indexes.sql` — creates all views and indexes.
5. `artconnect_procedures.sql` — creates all functions and procedures.
6. `artconnect_transactions.sql` — optional: runs the transactional demo scenarios.

Application Launch

```
cd ArtConnectPro-App && mvn javafx:run
```

Requires Java 23, Maven, and a running MySQL 8 instance. Configure credentials in `src/main/resources/database.properties`.